

Протокол

Автор: Андрій Білий <https://github.com/bilyilandrii>

GitHub (micro_hazelcast): [Проект](#)

Налаштування

Спочатку я підключив logging-service до кластеру Hazelcast. Це робиться схожим чином як було зі скриптами, створюю клієнта і вже через нього отримую доступ до Distributed Map.

logging_service/main.py:

```
client = HazelcastClient(  
    cluster_name="dev-cluster",  
    cluster_members=["hazelcast1:5701", "hazelcast2:5701",  
    "hazelcast3:5701"],  
)  
app.state.hz_client = client  
app.state.distributed_map = client.get_map("messages_map").blocking()
```

Я використовуватиму запити до Hz синхронно, оскільки пайтонівський Future не поєднується з тим що вертає hz-клієнт (а імплементація з asyncio є лише в beta). Це не сильно впливатиме на продуктивність, тому що FastAPI кидатиме всі синхронні запити на окремий потік, тобто load balancing відбуватиметься.

Кожен logging service ініціалізується через docker-compose.yml:

```
logging1:  
  image: logging-service:latest  
  build:  
    context: .  
    dockerfile: logging_service/Dockerfile  
  depends_on:  
    hazelcast1:  
      condition: service_healthy  
    hazelcast2:  
      condition: service_healthy  
    hazelcast3:  
      condition: service_healthy  
# logging2, logging3 схожим чином
```

Я використовую health_check, аби кожен зі сервісів логування підключався тоді і лише тоді, коли кластер є повністю піднятий.

docker-compose.yml

```

hazelcast1:
  image: hazelcast/hazelcast:5.3
  ports:
    - "5701:5701"
  volumes:
    - ./hazelcast.yaml:/opt/hazelcast/config/hazelcast.yaml
  environment:
    - HZ_CLUSTERNAME=dev-cluster
    -
  JAVA_OPTS=-Dhazelcast.config=/opt/hazelcast/config/hazelcast.yaml
  healthcheck:
    test:
      [
        "CMD",
        "curl",
        "-sf",
        "http://127.0.0.1:5701/hazelcast/health/node-state",
      ]
    interval: 10s
    timeout: 5s
    retries: 5
# hazelcast2, hazelcast3 СХОЖИМ ЧИНОМ

```

Типово, REST API у Hz є вимкненим, це пов'язано з питанням безпеки. Тому, щоб скористатися вбудованим механізмом перевірки здоров'я, треба явно це вказати:

hazelcast.yaml

```

hazelcast:
  cluster-name: dev-cluster
  network:
    join:
      tcp-ip:
        enabled: true
        member-list:
          - hazelcast1
          - hazelcast2
          - hazelcast3

  rest-api:
    enabled: true
    endpoint-groups:
      HEALTH_CHECK:
        enabled: true

```

Для counter service я використовую PostgreSQL (postgres:15-alpine image). Ініціалізація також відбувається через docker-compose.yml:

```
postgres_db:
  image: postgres:15-alpine
  environment:
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
    POSTGRES_DB: counter_db
  ports:
    - "5432:5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U user -d counter_db"]
    interval: 5s
    timeout: 5s
    retries: 5

counter_service:
  image: counter-service:latest
  build:
    context: .
    dockerfile: counter_service/Dockerfile
  environment:
    - DB_URL=postgresql://user:password@postgres_db:5432/counter_db
  depends_on:
    postgres_db:
      condition: service_healthy
```

Тут також використовую health_check аби counter міг акуратно підключитися. Для перевірки здоров'я існує вбудована команда pg_isready, яка каже, чи база готова приймати нові з'єднання.

В коді це виглядатиме якимось так:

```
pool = ThreadedConnectionPool(
    minconn=1,
    maxconn=20,
    dsn=DB_URL,
)
app.state.db_pool = pool

conn = pool.getconn()
try:
    with conn:
        with conn.cursor() as cur:
            cur.execute("""
```


Крім того всього, я покращив /health ендпоінт і тепер він перевіряє чи сервіси змогли доєднатися до своїх сховищ:

Curl

```
curl -X 'GET' \
  'http://localhost:8080/health' \
  -H 'accept: application/json'
```

Бачимо, що все добре:

Code	Details
200	Response body <pre>{ "status": "ok", "counter_service": { "status": "ok", "database": "connected" }, "logging_services": { "http://logging1:8000": { "status": "ok", "hazelcast": "connected" }, "http://logging2:8000": { "status": "ok", "hazelcast": "connected" }, "http://logging3:8000": { "status": "ok", "hazelcast": "connected" } } }</pre>

Базовий Функціонал

Спочатку роблю 10 різних транзакцій на id “user”.

POST-Запити та Відповіді

Нижче наведено які саме повідомлення надсилались на facade service.

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": 10 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "1774026265174766131", "balance": 10 }</pre>

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": 200 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "1774026299571376133", "balance": 210 }</pre>

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": -23 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "1774026329056448550", "balance": 187 }</pre>

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": 400 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "1774026367048821970", "balance": 587 }</pre>

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": -333 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "1774026394722699927", "balance": 254 }</pre>

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": 1000 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "1774026418786624633", "balance": 1254 }</pre>

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": -555 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "177402644216955546", "balance": 699 }</pre>

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": 888 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "1774026461360110680", "balance": 1587 }</pre>

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": -222 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "1774026494192944792", "balance": 1365 }</pre>

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user", "amount": 333 }'</pre>	
Request URL	
http://localhost:8080/process	
Server response	
Code	Details
200	Response body <pre>{ "transaction_id": "1774026523257657709", "balance": 1598 }</pre>

Розподіл Запитів

logging!

```
[LOGGING] Connected to Hazelcast cluster!
INFO      Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
INFO      172.18.0.11:35316 - "GET /health HTTP/1.1" 200
[LOGGING] Stored transaction 1774026265174766131 for user 'user'
INFO      172.18.0.11:50178 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1774026299571376133 for user 'user'
INFO      172.18.0.11:46920 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1774026394722699927 for user 'user'
INFO      172.18.0.11:36240 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1774026418786624633 for user 'user'
INFO      172.18.0.11:50460 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1774026461360110680 for user 'user'
INFO      172.18.0.11:40412 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1774026494192944792 for user 'user'
INFO      172.18.0.11:33234 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1774026523257657709 for user 'user'
INFO      172.18.0.11:46394 - "POST /transaction HTTP/1.1" 200
```

logging2:

```
[LOGGING] Connected to Hazelcast cluster!
INFO Application startup complete.
INFO Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO 172.18.0.11:39198 - "GET /health HTTP/1.1" 200
[LOGGING] Stored transaction 1774026329056448550 for user 'user'
INFO 172.18.0.11:40848 - "POST /transaction HTTP/1.1" 200
```

logging3:

```
[LOGGING] Connected to Hazelcast cluster!
INFO Application startup complete.
INFO Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO 172.18.0.11:58964 - "GET /health HTTP/1.1" 200
[LOGGING] Stored transaction 1774026367048821970 for user 'user'
INFO 172.18.0.11:41898 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1774026442169555546 for user 'user'
INFO 172.18.0.11:51726 - "POST /transaction HTTP/1.1" 200
```

Як можемо побачити не всі запити йдуть на один сервіс, 10 - це доволі мала вибірка тому розподіл може бути не настільки рівномірним, але на більших числах це буде помітніше.

GET-Запити та Відповіді

Окремий користувач:

Curl

```
curl -X 'GET' \
'http://localhost:8080/user/user' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8080/user/user
```

Server response

Code	Details
200	Response body <pre>{ "balance": 1698, "transactions": [{ "user_id": "user", "amount": 10, "transaction_id": "1774026265174766131" }, { "user_id": "user", "amount": -23, "transaction_id": "1774026329056448550" }, { "user_id": "user", "amount": -555, "transaction_id": "1774026442169555546" }, { "user_id": "user", "amount": 1000, "transaction_id": "1774026418786624633" }, { "user_id": "user", </pre>

```
{
  "user_id": "user",
  "amount": -333,
  "transaction_id": "1774026394722699927"
},
{
  "user_id": "user",
  "amount": -222,
  "transaction_id": "1774026494192944792"
},
{
  "user_id": "user",
  "amount": 200,
  "transaction_id": "1774026299571376133"
},
{
  "user_id": "user",
  "amount": 333,
  "transaction_id": "1774026523257657709"
},
{
  "user_id": "user",
  "amount": 400,
  "transaction_id": "1774026367048821970"
}
```

```
,
{
  "user_id": "user",
  "amount": 888,
  "transaction_id": "1774026461360110680"
}
]
```

Всі акаунти:

Curl	
<pre>curl -X 'GET' \ 'http://localhost:8080/accounts' \ -H 'accept: application/json'</pre>	
Request URL	
<pre>http://localhost:8080/accounts</pre>	
Server response	
Code	Details
200	Response body <pre>{ "user": 1698 }</pre>

Бачимо, що запити були збережені правильно і в Distributed Map, і в Postgres.

Перевірка Відмовостійкості

Logging:

Спочатку хочу перевірити те, чи система продовжить працювати, якщо відключити один-два екземпляри сервісів логування.

Зупиняємо контейнер.

```
docker compose stop logging3
```

Після запиту на /health, можна побачити що сервіс справді впав.

Code	Details
200	Response body <pre>{ "status": "ok", "counter_service": { "status": "ok", "database": "connected" }, "logging_services": { "http://logging1:8000": { "status": "ok", "hazelcast": "connected" }, "http://logging2:8000": { "status": "ok", "hazelcast": "connected" }, "http://logging3:8000": "down" } }</pre>

Роблю два простенькі запити на новий акаунт:

Curl

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_test",
    "amount": 200
  }'
```

Request URL

`http://localhost:8080/process`

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1774029652269356804", "balance": 200 }</pre>

Curl

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_test",
    "amount": -150
  }'
```

Request URL

`http://localhost:8080/process`

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1774029677947500885", "balance": 50 }</pre>

Всі транзакції збереглися:

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user_test' \
  -H 'accept: application/json'
```

Request URL

`http://localhost:8080/user/user_test`

Server response

Code	Details
200	Response body <pre>{ "balance": 50, "transactions": [{ "user_id": "user_test", "amount": 200, "transaction_id": "1774029652269356804" }, { "user_id": "user_test", "amount": -150, "transaction_id": "1774029677947500885" }] }</pre>

А наші запити розподілилися лише між logging-1 та logging-2.

logging1:

```
INFO    172.18.0.11:33326 - "GET /health HTTP/1.1" 200
[LOGGING] Stored transaction 1774029677947500885 for user 'user_test'
INFO    172.18.0.11:49896 - "POST /transaction HTTP/1.1" 200
```

logging2:

```
[LOGGING] Stored transaction 1774029652269356804 for user 'user_test'
INFO    172.18.0.11:41568 - "POST /transaction HTTP/1.1" 200
INFO    172.18.0.11:45938 - "GET /transaction/user_test HTTP/1.1" 200
```

Тепер спробуємо зупинити другий:

```
docker compose stop logging2
```

Sanity check:

Code	Details
200	Response body<pre>{ "status": "ok", "counter_service": { "status": "ok", "database": "connected" }, "logging_services": { "http://logging1:8000": { "status": "ok", "hazelcast": "connected" }, "http://logging2:8000": "down", "http://logging3:8000": "down" } }</pre>

Роблю один простий запит (на той самий акк) і отримую аналогічну відповідь (справа)

Curl	
<pre>curl -X 'POST' \ 'http://localhost:8080/process' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "user_id": "user_test", "amount": 333 }'</pre>	
Request URL	
<code>http://localhost:8080/process</code>	
Server response	
Code	Details
200	Response body<pre>{ "transaction_id": "1774029914254569258", "balance": 383 }</pre>

Curl	
<pre>curl -X 'GET' \ 'http://localhost:8080/user/user_test' \ -H 'accept: application/json'</pre>	
Request URL	
<code>http://localhost:8080/user/user_test</code>	
Server response	
Code	Details
200	Response body<pre>{ "balance": 383, "transactions": [{ "user_id": "user_test", "amount": 200, "transaction_id": "1774029652269356804" }, { "user_id": "user_test", "amount": 333, "transaction_id": "1774029914254569258" }, { "user_id": "user_test", "amount": -150, "transaction_id": "1774029677947500885" }] }</pre>

Спостерігаємо, що всі запити були направлені на один сервіс логування:

```
[LOGGING] Stored transaction 1774029914254569258 for user 'user_test'
INFO    172.18.0.11:42784 - "POST /transaction HTTP/1.1" 200
INFO    172.18.0.11:43384 - "GET /health HTTP/1.1" 200
INFO    172.18.0.11:38196 - "GET /transaction/user_test HTTP/1.1" 200
```

Hazelcast:

Ми знаємо що є записані транзакцій двох попередніх користувачів. Вони розподілені таким чином:

Member ^	^ Entries	^ Gets	^ Puts	^ Removals	^ Sets	^ Entry Memory
hazelcast1:5701	2	0	1	0	0	467.00 B
hazelcast2:5701	8	0	9	0	0	1.84 kB
hazelcast3:5701	3	0	3	0	0	708.00 B
TOTAL	13	0	13	0	0	2.99 kB

Схожим чином зупиняю ноди:

```
docker compose stop hazelcast3
```

Після відключення можна зауважити, що запити були перерозподілені:

Member ^	^ Entries	^ Gets	^ Puts	^ Removals	^ Sets	^ Entry Memory
hazelcast1:5701	2	0	1	0	0	467.00 B
hazelcast2:5701	11	0	9	0	0	2.53 kB
hazelcast3:5701	N/A	N/A	N/A	N/A	N/A	N/A
TOTAL	13	0	10	0	0	2.99 kB

І в логах однієї з нод відбулася зміна розміру кластеру:

```
Members {size:2, ver:4} [
  Member [hazelcast1]:5701 - e082aa65-e0a4-4e23-87ab-8a3a9e03cd32 this
  Member [hazelcast2]:5701 - 679237e8-8be7-409b-a368-83437af90c24
]

2026-03-20 18:14:09,666 [ INFO] [hz.elated_euler.cached.thread-5] [c.h.t.TransactionManagerService]: [hazelcast1]:5701
[dev-cluster] [5.3.8] Committing/rolling-back live transactions of [hazelcast3]:5701, UUID: 8b515082-0161-45f8-92a0-
68489b9e8e1d
2026-03-20 18:14:09,718 [ INFO] [hz.elated_euler.migration] [c.h.i.p.i.MigrationManager]: [hazelcast1]:5701 [dev-cluster]
[5.3.8] Repartitioning cluster data. Migration tasks count: 180
2026-03-20 18:14:09,796 [ INFO] [hz.elated_euler.migration] [c.h.i.p.i.MigrationManager]: [hazelcast1]:5701 [dev-cluster]
[5.3.8] All migration tasks have been completed. (repartitionTime=Fri Mar 20 18:14:09 GMT 2026, plannedMigrations=180,
completedMigrations=180, remainingMigrations=0, totalCompletedMigrations=180)
2026-03-20 18:14:30,480 [ WARN] [hz.elated_euler.cached.thread-10] [c.h.i.c.i.TcpIpJoiner]: [hazelcast1]:5701 [dev-cluster]
[5.3.8] Cannot resolve hostname 'hazelcast3'. Please make sure host is valid and reachable.
```

Спочатку перевіримо, чи попередні записи точно збереглись:

```
Curl
curl -X 'GET' \
  'http://localhost:8080/user/user_test' \
  -H 'accept: application/json'

Request URL
http://localhost:8080/user/user_test

Server response

Code    Details
200     Response body
{
  "balance": 383,
  "transactions": [
    {
      "user_id": "user_test",
      "amount": -159,
      "transaction_id": "1774029677947500885"
    },
    {
      "user_id": "user_test",
      "amount": 200,
      "transaction_id": "1774029652269356804"
    },
    {
      "user_id": "user_test",
      "amount": 333,
      "transaction_id": "1774029914254569258"
    }
  ]
}
```

```
Code    Details
200     Response body
{
  "balance": 1698,
  "transactions": [
    {
      "user_id": "user",
      "amount": 10,
      "transaction_id": "1774026265174766131"
    },
    {
      "user_id": "user",
      "amount": -23,
      "transaction_id": "1774026329056448550"
    },
    {
      "user_id": "user",
      "amount": -555,
      "transaction_id": "1774026442169555546"
    },
    {
      "user_id": "user",
      "amount": 1000,
      "transaction_id": "1774026418786624633"
    },
    {
      "user_id": "user"
    }
  ]
}
```

```
{
  "user_id": "user",
  "amount": 400,
  "transaction_id": "1774026367048821970"
},
{
  "user_id": "user",
  "amount": -333,
  "transaction_id": "1774026394722699927"
},
{
  "user_id": "user",
  "amount": -222,
  "transaction_id": "1774026494192944792"
},
{
  "user_id": "user",
  "amount": 888,
  "transaction_id": "1774026461360110680"
},
{
  "user_id": "user"
}
```

```
,
{
  "user_id": "user",
  "amount": 200,
  "transaction_id": "1774026299571376133"
},
{
  "user_id": "user",
  "amount": 333,
  "transaction_id": "1774026523257657709"
}
]
```

Бачимо, що всі 13 транзакцій збереглись. Тепер спробуємо додати одну:

```
Curl
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_test2",
    "amount": 555
  }'

Request URL
http://localhost:8080/process

Server response

Code    Details
200     Response body
{
  "transaction_id": "1774030636263474676",
  "balance": 555
}

Response headers
content-length: 56
content-type: application/json
date: Fri, 20 Mar 2026 18:17:15 GMT
server: uvicorn
```

Перш ніж перевіряти, я зупиню ще одну ноду (щоб було цікавіше):

```
docker compose stop hazelcast2
```

Знову бачимо, що перерозподіляються дані:

Member ^	Entries ^	Gets ^	Puts ^	Removals ^	Sets ^	Entry Memory ^
hazelcast1:5701	14	0	2	0	0	3.22 kB
hazelcast3:5701	N/A	N/A	N/A	N/A	N/A	N/A
TOTAL	14	0	2	0	0	3.22 kB

Розмір кластеру також зменшився:

```
Members {size:1, ver:5} [
  Member [hazelcast1]:5701 - e082aa65-e0a4-4e23-87ab-8a3a9e03cd32 this
]

2026-03-20 18:22:27,479 [ INFO] [hz.elated_euler.cached.thread-7] [c.h.t.TransactionManagerService]: [hazelcast1]:5701
[dev-cluster] [5.3.8] Committing/rolling-back live transactions of [hazelcast2]:5701, UUID: 679237e8-8be7-409b-a368-
83437af90c24
2026-03-20 18:22:27,535 [ INFO] [hz.elated_euler.migration] [c.h.i.p.i.MigrationManager]: [hazelcast1]:5701 [dev-cluster]
[5.3.8] Partition balance is ok, no need to repartition.
2026-03-20 18:22:30,481 [ WARN] [hz.elated_euler.cached.thread-5] [c.h.i.c.i.TcpIpJoiner]: [hazelcast1]:5701 [dev-cluster]
[5.3.8] Cannot resolve hostname 'hazelcast3'. Please make sure host is valid and reachable.
```

І тепер знову перевіримо чи точно усі дані можна дістати:

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user_test2' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/user/user_test2
```

Server response

Code	Details
200	Response body <pre>{ "balance": 555, "transactions": [{ "user_id": "user_test2", "amount": 555, "transaction_id": "1774030636263474676" }] }</pre>

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user_test' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/user/user_test
```

Server response

Code	Details
200	Response body <pre>{ "balance": 383, "transactions": [{ "user_id": "user_test", "amount": -150, "transaction_id": "1774029677947500885" }, { "user_id": "user_test", "amount": 200, "transaction_id": "1774029652269356804" }, { "user_id": "user_test", "amount": 333, "transaction_id": "1774029914254569258" }] }</pre>

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/user/user
```

Server response

Code	Details
200	Response body <pre>{ "balance": 1698, "transactions": [{ "user_id": "user", "amount": -555, "transaction_id": "1774026442169555546" }, { "user_id": "user", "amount": 10, "transaction_id": "1774026265174766131" }, { "user_id": "user", "amount": -23, "transaction_id": "1774026329856448558" }, { "user_id": "user", "amount": 1000, "transaction_id": "1774026418786624633" }], "user_id": "user" }</pre>

```
{
  "user_id": "user",
  "amount": 400,
  "transaction_id": "1774026367048821970"
},
{
  "user_id": "user",
  "amount": -333,
  "transaction_id": "1774026394722699927"
},
{
  "user_id": "user",
  "amount": -222,
  "transaction_id": "1774026494192944792"
},
{
  "user_id": "user",
  "amount": 888,
  "transaction_id": "1774026461360110680"
},
{
  "user_id": "user",
  "amount": 200,
  "transaction_id": "1774026299571376133"
},
{
  "user_id": "user",
  "amount": 333,
  "transaction_id": "1774026523257657709"
}
```

```
{
  "user_id": "user",
  "amount": 333,
  "transaction_id": "1774026523257657709"
}
```

Всі 14 транзакцій справді існують!

Ну і ще перевіримо, чи можна здійснювати нові транзакції:

Curl

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_test3",
    "amount": 228
  }'
```

Request URL

```
http://localhost:8080/process
```

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1774031102635091878", "balance": 228 }</pre>

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user_test3' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/user/user_test3
```

Server response

Code	Details
200	Response body <pre>{ "balance": 228, "transactions": [{ "user_id": "user_test3", "amount": 228, "transaction_id": "1774031102635091878" }] }</pre>

Логи інших мікросервісів facade:

```
[FACADE] Received request from user 'user' for amount 10.0
[FACADE] Completed transaction 1774026265174766131. Final balance: 10.0
INFO 192.168.65.1:46790 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 200.0
[FACADE] Completed transaction 1774026299571376133. Final balance: 210.0
INFO 192.168.65.1:19447 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount -23.0
[FACADE] Completed transaction 1774026329056448550. Final balance: 187.0
INFO 192.168.65.1:31191 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 400.0
[FACADE] Completed transaction 1774026367048821970. Final balance: 587.0
INFO 192.168.65.1:28649 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount -333.0
[FACADE] Completed transaction 1774026394722699927. Final balance: 254.0
INFO 192.168.65.1:51259 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 1000.0
[FACADE] Completed transaction 1774026418786624633. Final balance: 1254.0
INFO 192.168.65.1:54537 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount -555.0
[FACADE] Completed transaction 1774026442169555546. Final balance: 699.0
INFO 192.168.65.1:57321 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 888.0
[FACADE] Completed transaction 1774026461360110680. Final balance: 1587.0
INFO 192.168.65.1:61110 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount -222.0
[FACADE] Completed transaction 1774026494192944792. Final balance: 1365.0
INFO 192.168.65.1:23431 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 333.0
[FACADE] Completed transaction 1774026523257657709. Final balance: 1698.0
INFO 192.168.65.1:17228 - "POST /process HTTP/1.1" 200
INFO 192.168.65.1:25132 - "GET /user/user HTTP/1.1" 200
INFO 192.168.65.1:37483 - "GET /accounts HTTP/1.1" 200
INFO 192.168.65.1:38282 - "GET /health HTTP/1.1" 200
```

```
[FACADE] Received request from user 'user_test' for amount 200.0
[FACADE] Completed transaction 1774029652269356804. Final balance: 200.0
INFO 192.168.65.1:49714 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user_test' for amount -150.0
[FACADE] Completed transaction 1774029677947500885. Final balance: 50.0
INFO 192.168.65.1:56968 - "POST /process HTTP/1.1" 200
INFO 192.168.65.1:23864 - "GET /user/user_test HTTP/1.1" 200
[FACADE] Received request from user 'user_test' for amount 333.0
[FACADE] Failed to connect to http://logging2:8080, falling back to next instance...
[FACADE] Failed to connect to http://logging3:8080, falling back to next instance...
[FACADE] Completed transaction 1774029914254569258. Final balance: 383.0
INFO 192.168.65.1:34867 - "POST /process HTTP/1.1" 200
INFO 192.168.65.1:24761 - "GET /health HTTP/1.1" 200
INFO 192.168.65.1:46032 - "GET /user/user_test HTTP/1.1" 200
INFO 192.168.65.1:45371 - "GET /user/user_test HTTP/1.1" 200
[FACADE] Received request from user 'user_test2' for amount 555.0
[FACADE] Completed transaction 1774030636263474676. Final balance: 555.0
INFO 192.168.65.1:21835 - "POST /process HTTP/1.1" 200
INFO 192.168.65.1:16722 - "GET /user/user_test2 HTTP/1.1" 200
INFO 192.168.65.1:19426 - "GET /user/user_test HTTP/1.1" 200
INFO 192.168.65.1:60401 - "GET /user/user HTTP/1.1" 200
INFO 192.168.65.1:54398 - "GET /user/user_test2 HTTP/1.1" 200
INFO 192.168.65.1:62264 - "GET /user/user_test HTTP/1.1" 200
[FACADE] Received request from user 'user_test3' for amount 228.0
[FACADE] Completed transaction 1774031102635091878. Final balance: 228.0
INFO 192.168.65.1:54062 - "POST /process HTTP/1.1" 200
INFO 192.168.65.1:34832 - "GET /user/user_test3 HTTP/1.1" 200
INFO 192.168.65.1:22527 - "GET /user/user HTTP/1.1" 200
INFO 192.168.65.1:40073 - "GET /accounts HTTP/1.1" 200
```

Також хочу зауважити, що коли сервіс логування недоступний, то facade пробує доступитися до інших. На першій фотці бачимо що не прийшлося перебирати, але вже на другій (відключили два екземпляри) фасад пробував доступитися спочатку до логінгу 2, потім 3, і тільки перший відповів (все, як планувалось).

counter:

```
INFO Waiting for application startup.
[COUNTER] Successfully connected to PostgreSQL!
INFO Application startup complete.
INFO Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO 172.18.0.11:60718 - "GET /health HTTP/1.1" 200
[COUNTER] User 'user' applied 10.0. New balance: 10.00
INFO 172.18.0.11:52520 - "POST /transaction HTTP/1.1" 200
[COUNTER] User 'user' applied 200.0. New balance: 210.00
INFO 172.18.0.11:52638 - "POST /transaction HTTP/1.1" 200
[COUNTER] User 'user' applied -23.0. New balance: 187.00
INFO 172.18.0.11:55014 - "POST /transaction HTTP/1.1" 200
[COUNTER] User 'user' applied 400.0. New balance: 587.00
INFO 172.18.0.11:49492 - "POST /transaction HTTP/1.1" 200
[COUNTER] User 'user' applied -333.0. New balance: 254.00
INFO 172.18.0.11:48978 - "POST /transaction HTTP/1.1" 200
[COUNTER] User 'user' applied 1000.0. New balance: 1254.00
INFO 172.18.0.11:47840 - "POST /transaction HTTP/1.1" 200
[COUNTER] User 'user' applied -555.0. New balance: 699.00
INFO 172.18.0.11:49334 - "POST /transaction HTTP/1.1" 200
[COUNTER] User 'user' applied 888.0. New balance: 1587.00
INFO 172.18.0.11:49332 - "POST /transaction HTTP/1.1" 200
[COUNTER] User 'user' applied -222.0. New balance: 1365.00
INFO 172.18.0.11:34900 - "POST /transaction HTTP/1.1" 200
[COUNTER] User 'user' applied 333.0. New balance: 1698.00
INFO 172.18.0.11:44038 - "POST /transaction HTTP/1.1" 200
INFO 172.18.0.11:54666 - "GET /balance/user HTTP/1.1" 200
INFO 172.18.0.11:36294 - "GET /balance HTTP/1.1" 200
INFO 172.18.0.11:58882 - "GET /health HTTP/1.1" 200
[COUNTER] User 'user_test' applied 200.0. New balance: 200.00
INFO 172.18.0.11:40238 - "POST /transaction HTTP/1.1" 200
```

```
[COUNTER] User 'user_test' applied -150.0. New balance: 50.00
INFO 172.18.0.11:59486 - "POST /transaction HTTP/1.1" 200
INFO 172.18.0.11:50798 - "GET /balance/user_test HTTP/1.1" 200
[COUNTER] User 'user_test' applied 333.0. New balance: 383.00
INFO 172.18.0.11:40868 - "POST /transaction HTTP/1.1" 200
INFO 172.18.0.11:54778 - "GET /health HTTP/1.1" 200
INFO 172.18.0.11:55236 - "GET /balance/user_test HTTP/1.1" 200
INFO 172.18.0.11:60386 - "GET /balance/user_test HTTP/1.1" 200
[COUNTER] User 'user_test2' applied 555.0. New balance: 555.00
INFO 172.18.0.11:40970 - "POST /transaction HTTP/1.1" 200
INFO 172.18.0.11:50706 - "GET /balance/user_test2 HTTP/1.1" 200
INFO 172.18.0.11:45590 - "GET /balance/user_test HTTP/1.1" 200
INFO 172.18.0.11:60136 - "GET /balance/user HTTP/1.1" 200
INFO 172.18.0.11:56456 - "GET /balance/user_test2 HTTP/1.1" 200
INFO 172.18.0.11:34316 - "GET /balance/user_test HTTP/1.1" 200
[COUNTER] User 'user_test3' applied 228.0. New balance: 228.00
INFO 172.18.0.11:54670 - "POST /transaction HTTP/1.1" 200
INFO 172.18.0.11:45294 - "GET /balance/user_test3 HTTP/1.1" 200
INFO 172.18.0.11:33842 - "GET /balance/user HTTP/1.1" 200
INFO 172.18.0.11:53422 - "GET /balance HTTP/1.1" 200
```

Дані справді записуються до бази:

```
2026-03-20 17:08:54.226 UTC [55] LOG: checkpoint complete: wrote 88 buffers (0.5%); 0 WAL file(s) added, 0 removed, 0 recycled; write=8.730 s, sync=0.008 s, total=8.749 s; sync files=49, longest=0.003 s, average=0.001 s; distance=404 kB, estimate=404 kB
2026-03-20 18:04:29.694 UTC [55] LOG: checkpoint starting: time
2026-03-20 18:04:29.910 UTC [55] LOG: checkpoint complete: wrote 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.206 s, sync=0.004 s, total=0.216 s; sync files=3, longest=0.003 s, average=0.002 s; distance=1 kB, estimate=363 kB
2026-03-20 18:09:29.981 UTC [55] LOG: checkpoint starting: time
2026-03-20 18:09:30.100 UTC [55] LOG: checkpoint complete: wrote 2 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.108 s, sync=0.004 s, total=0.120 s; sync files=2, longest=0.003 s, average=0.002 s; distance=0 kB, estimate=327 kB
2026-03-20 18:19:29.183 UTC [55] LOG: checkpoint starting: time
2026-03-20 18:19:29.404 UTC [55] LOG: checkpoint complete: wrote 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.205 s, sync=0.005 s, total=0.222 s; sync files=3, longest=0.004 s, average=0.002 s; distance=1 kB, estimate=294 kB
2026-03-20 18:29:29.495 UTC [55] LOG: checkpoint starting: time
2026-03-20 18:29:29.712 UTC [55] LOG: checkpoint complete: wrote 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.205 s, sync=0.005 s, total=0.217 s; sync files=3, longest=0.004 s, average=0.002 s; distance=1 kB, estimate=265 kB
```


Тестування Продуктивності

Зліва можете побачити оновлену версію і справа результати попередньої.

```
andrij@MyPookie Hws % uv run perf_test.py

=====
Starting Scenario 1 (10 Clients, 10 Unique Accounts)
=====

--- Performance Results ---
Total Requests: 100000
Total Time: 200.79 seconds
Requests Per Second (RPS): 498.03

--- Internal Time Contributions ---
Logging Service Total Delay: 493.78s
Counter Service Total Delay: 1556.05s

--- Balance Verification ---
user_0 Balance: 10000.0 (Expected: 10000.0)
user_1 Balance: 10000.0 (Expected: 10000.0)
user_2 Balance: 10000.0 (Expected: 10000.0)
user_3 Balance: 10000.0 (Expected: 10000.0)
user_4 Balance: 10000.0 (Expected: 10000.0)
user_5 Balance: 10000.0 (Expected: 10000.0)
user_6 Balance: 10000.0 (Expected: 10000.0)
user_7 Balance: 10000.0 (Expected: 10000.0)
user_8 Balance: 10000.0 (Expected: 10000.0)
user_9 Balance: 10000.0 (Expected: 10000.0)
```

```
=====
Starting Scenario 1 (10 Clients, 10 Unique Accounts)
=====

--- Performance Results ---
Total Requests: 100000
Total Time: 173.96 seconds
Requests Per Second (RPS): 574.83

--- Internal Time Contributions ---
Logging Service Total Delay: 1014.24s
Counter Service Total Delay: 991.39s

--- Balance Verification ---
user_0 Balance: 10000 (Expected: 10000)
user_1 Balance: 10000 (Expected: 10000)
user_2 Balance: 10000 (Expected: 10000)
user_3 Balance: 10000 (Expected: 10000)
user_4 Balance: 10000 (Expected: 10000)
user_5 Balance: 10000 (Expected: 10000)
user_6 Balance: 10000 (Expected: 10000)
user_7 Balance: 10000 (Expected: 10000)
user_8 Balance: 10000 (Expected: 10000)
user_9 Balance: 10000 (Expected: 10000)
```

```
=====
Starting Scenario 2 (10 Clients, 1 Shared Account)
=====

--- Performance Results ---
Total Requests: 100000
Total Time: 205.22 seconds
Requests Per Second (RPS): 487.27

--- Internal Time Contributions ---
Logging Service Total Delay: 525.40s
Counter Service Total Delay: 1597.94s

--- Balance Verification ---
Shared Account Balance: 100000.0 (Expected: 100000.0)
```

```
=====
Starting Scenario 2 (10 Clients, 1 Shared Account)
=====

--- Performance Results ---
Total Requests: 100000
Total Time: 174.48 seconds
Requests Per Second (RPS): 573.13

--- Internal Time Contributions ---
Logging Service Total Delay: 1011.81s
Counter Service Total Delay: 993.84s

--- Balance Verification ---
Shared Account Balance: 100000 (Expected: 100000)
```

Розподіл даних по нодах Hz (1 запит був тестовим):

Map Statistics (In-Memory Format: BINARY)								
		RESET TIME	1 minute ago	→	now	Default View		 
Member ^	Entries	Gets	Puts	Removals	Sets	Entry Memory	^ E	
hazelcast1:5701	66,536	0	66,536	0	0	14.85 MB		
hazelcast2:5701	66,646	0	66,646	0	0	14.87 MB		
hazelcast3:5701	66,819	0	66,819	0	0	14.91 MB		
TOTAL	200,001	0	200,001	0	0	44.63 MB		

Найбільше, що кидається очі - це покращення швидкості в сервісі логування. Попередньо це було ≈ 1000 с, зараз delay є приблизно вдвічі меншим. Хоч і в теперішній імплементації з'являються додаткові запити тепер на кластер Hz, але load balancing через три екземпляри дає про себе знати. Навіть якщо попередня імплементація і використовувала швидку локальну таблицю, але вона з часом переповнювалась і збільшувався відповідно час на додавання нових даних. Тим більше всі 100к запитів оброблялися лише одним сервісом, коли це робить тепер три.

Що не скажеш насправді про сервіс підрахунку (було ≈ 992 с, стало ≈ 1577 с), оскільки ми перейшли до реляційної бази даних (на відміну від локальної таблиці), то у нас з'являється купа додаткових дій для того щоб зберегти/оновити баланс користувача. Тепер нам потрібно додатково робити запити до контейнера PostgreSQL, він, у свою чергу, має розпарсити цю Query, отримати lock (на рядок), оновити дані на диску, додатково зберегти їх у файл логів (щоб в разі чого відновити запис, ACID moment), і на кінець відправити результат назад по мережі. Тобто якщо колись у нас швидкість була обмежена мережею і RAM, то тепер з'являється ще більше мережі та дисковий I/O.

Також можна зауважити, що загальний RPS впав приблизно на 90 одиниць. Хоч ми і отримали набагато швидший сервіс логування, але час на counter відповідно збільшився. Оскільки фасад чекатиме поки обидва сервіси закінчать обробляти транзакцію, то загальний час буде характеризуватися її найповільнішою компонентою.

Незважаючи на це все, ніяких data races не відбулося і поточна імплементація працює коректно для такої кількості одночасних клієнтів. Плюс тепер ми маємо набагато краще забезпечення цілісності даних, тому що Hz має backup-count=2 (через hazelcast.yaml) і навіть якщо один чи два вузла впаде, то дані не будуть втрачені. Також ми можемо зберігати набагато більше користувачів, оскільки є реляційна bd. І звичайно великим плюсом є те, що тепер система ще більш fault-tolerance, оскільки використовуємо декілька екземплярів сервісу логування.

Висновок

На загал, в цій домашці вдалося поєднати результати першої та другої і отримати бажаний результат для сервісу логування. Але загальний час на опрацювання запитів збільшився, оскільки тепер використовується реляційне сховище. Відчуваю що в наступній роботі будемо працювати саме над цим (реплікації і load balancing для counter service).